Numerical Method - Homework 2

B13902022 Yu-Chi,Lai

1

1.1 (a)

In section 3.4, I found: "Trivially, you can also apply a positive operator to a numeric expression: `int x = +42;` Numeric values are assumed to be positive unless explicitly made negative, so this operator has no effect on program operation"

Hence, the expression 0.0 conveys +0.0 by default. The code and its output is my experiment.

```c
#include <stdio.h>
#include <stdint.h>
#include <string.h>

void print_bits(double d, const char* label) {
    uint64_t bits;
    memcpy(&bits, &d, sizeof(bits));
    printf("%s: %f | Raw Hex: 0x%016lx\n", label, d, bits);
}

int main() {
    double pos_zero = 0.0, neg_zero = -0.0;

    print_bits(pos_zero, "Literal 0.0   ");
    print_bits(neg_zero, "Literal -0.0  ");

    if (pos_zero == neg_zero) {
        printf("\nResult: The == operator says they are equal.\n");
    }

    printf("1 /  0.0 = %f\n", 1.0 / pos_zero);
    printf("1 / -0.0 = %f\n", 1.0 / neg_zero);

    return 0;
}
```

```
Literal 0.0   : 0.000000 | Raw Hex: 0x0000000000000000
Literal -0.0  : -0.000000 | Raw Hex: 0x8000000000000000

Result: The == operator says they are equal.
1 /  0.0 = inf
1 / -0.0 = -inf
```

Figure 1: The output

1.2 (b)

In section 7.12.3.6, there's an `signbit()` macro in C99 standard (And you should also `#include <math.h>`). The `signbit` macro returns a nonzero value if and only if the sign of its argument value is negative.

1.3 (c)

We can declare/specify them using unary operators, positive/negative number division by infinity, zero times positive/negative number or the `copysign()` method in `<math.h>`. The program below is my experiment:

```c
#include <stdio.h>
#include <math.h>

int main() {
    double values[] = {-0.0, 0.0, -1.0 / INFINITY, 1.0 / INFINITY, -1.0 * 0.0, 1.0 * 0.0,
    ↪  copysign(0.0, 1.0), copysign(0.0, -1.0)};

    const char *names[] = {"Literal -0.0", "Literal 0.0", "-1.0 / INFINITY", "1.0 /
    ↪  INFINITY", "-1.0 * 0.0", "1.0 * 0.0", "copysign(0, +)", "copysign(0, -)"};

    int n = sizeof(values) / sizeof(values[0]);

    printf("Method             | Value | signbit()\n");
    printf("-------------------|-------|----------\n");

    for (int i = 0; i < n; i++) {
        printf("%-18s | %4.1f  | %d\n", names[i], values[i], signbit(values[i]));
    }

    printf("\n--- Validation through Division ---\n");
    printf("1.0 / +0.0 = %f\n", 1.0 / values[1]);
    printf("1.0 / -0.0 = %f\n", 1.0 / values[0]);

    return 0;
}
```

```
Method             | Value | signbit()
-------------------|-------|----------
Literal -0.0       | -0.0  | 1
Literal 0.0        |  0.0  | 0
-1.0 / INFINITY    | -0.0  | 1
1.0 / INFINITY     |  0.0  | 0
-1.0 * 0.0         | -0.0  | 1
1.0 * 0.0          |  0.0  | 0
copysign(0, +)     |  0.0  | 0
copysign(0, -)     | -0.0  | 1

--- Validation through Division ---
1.0 / +0.0 = inf
1.0 / -0.0 = -inf
```

Figure 2: The output of the program above.

1.4 (d)

We should adopt the first implementation $((x > y)?x : y)$ because if we set $b = -0.0$, the returned value of the first implementation is +0.0=0.0 (Hence, $c = -\infty < 0$) while the second implementation gives -0.0 (Hence, $c = +\infty > 0$). We can use the program below to prove our claim.

```c
#include <stdio.h>
#include <float.h>

double max1(double x, double y) {
    return (x > y) ? x : y;
}

double max2(double x, double y) {
    return (x < y) ? y : x;
}

int main() {
    double a = -5, b = -0.0;
    double c1 = a / max1(b, 0.0);
    double c2 = a / max2(b, 0.0);
    printf("%f %f\n", c1, c2);
}
```

```
-inf inf
```

Figure 3: The output.

2

(i)

Because I don't have matlab locally, I use GNU octave to implement everything.
No loop (vectorized) version:

```
function L = chol_vectorized(A)
    n = size(A, 1);
    L = zeros(n);
    for k = 1:n
        alpha = A(k, k);
        L(k, k) = sqrt(alpha);
        if (k < n)
            v = A(k+1:n, k);
            L(k+1:n, k) = v / L(k, k);
            A(k+1:n, k+1:n) = A(k+1:n, k+1:n) - (v * v') / alpha;
        end
    end
end
```

One-level loop version:

```
function L = chol_onelevel(A)
    n = size(A, 1);
    L = zeros(n);
    for k = 1:n
        alpha = A(k, k);
        L(k, k) = sqrt(alpha);
        if k < n
            v = A(k+1:n, k);
            L(k+1:n,k) = v / L(k, k);
            for i = 1:n-k
                A(k+1:n, k+i) = A(k+1:n, k+i) - v * (v(i) / alpha);
            end
        end
    end
end
```

Two-level loop version:

```
function L = chol_twolevel(A)
    n = size(A, 1);
    L = zeros(n);
    for k = 1:n
        alpha = A(k, k);
        L(k, k) = sqrt(alpha);
        if k < n
            v = A(k+1:n, k);
            L(k+1:n, k) = v / L(k, k);
            for i = 1:n-k
                for j = 1:n-k
                    A(k+i, k+j) = A(k+i, k+j) - (v(i) * v(j)) / alpha;
                end
            end
        end
    end
end
```

(ii)

In my code, I generate a random $n \times n$ matrix X, and then we get a symmetry matrix XX^T which is also positive semi-definite. The reason why it is can be shown below:

$$\text{Let } A = XX^T$$
$$A^T = (XX^T)^T = (X^T)^T X^T = XX^T = A$$
$$\forall v \in \mathbb{R}^n, v \neq \vec{0}, v^T Av = v^T AA^T v = (A^T v)^T (A^T v) \geq 0$$

And then by adding a $n \times n$ identity matrix times a positive constant, we can ensure that $v^T(A + nI)v = v^T Av + nv^T Iv > 0$, i.e., positive definite, without breaking the symmetry. The below is my code:

```
function A = generate_spd(n)
    X = randn(n);
    A = X * X' + n * eye(n);
```

```
endfunction

```

```
octave:1> A=generate_spd(4)
A =

   14.3011    0.1379   -1.6811    1.7991
    0.1379    8.4403    0.4961    0.1849
   -1.6811    0.4961    4.3781   -0.5137
    1.7991    0.1849   -0.5137    6.5175

octave:2> B=chol_vectorized(A), C=chol_onelevel(A), D=chol_twolevel(A), ans=chol(A, "lower")
B =

   3.7817        0        0        0
   0.0365   2.9050        0        0
  -0.4445   0.1764   2.0370        0
   0.4757   0.0577  -0.1534   2.5029

C =

   3.7817        0        0        0
   0.0365   2.9050        0        0
  -0.4445   0.1764   2.0370        0
   0.4757   0.0577  -0.1534   2.5029

D =

   3.7817        0        0        0
   0.0365   2.9050        0        0
  -0.4445   0.1764   2.0370        0
   0.4757   0.0577  -0.1534   2.5029

ans =

   3.7817        0        0        0
   0.0365   2.9050        0        0
  -0.4445   0.1764   2.0370        0
   0.4757   0.0577  -0.1534   2.5029
```

Figure 4: The correctness of my three implementations of cholesky factorization (proved by built-in `chol()`)

(iii)

I used meow1 (with GPU) in CSIE workstation, and the code for testing and its result are shown as below. Considering the overall performance, the efficiency: vectorized version > one-loop version > two-loop version. (For $n = 4000$, the execution time of vectorized version is a little bit longer than that of one-loop version, maybe some environmental issues or something occured.)

```
n_values = [10, 100, 1000, 4000];

for n = n_values
    A = generate_spd(n);

    % Measure Vectorized Version
    tic;
    chol_vectorized(A);
    time_vec = toc;

    % Measure One-level Loop Version
    tic;
    chol_onelevel(A);
    time_one = toc;

    % Measure Two-level Loop Version
    tic;
    chol_twolevel(A);
    time_two = toc;

    fprintf('For n=%d: Vectorized = %f s, One-loop = %f s, Two-loop = %f s\n', n, time_vec,
    ↪  time_one, time_two);
```

```
22 end
```

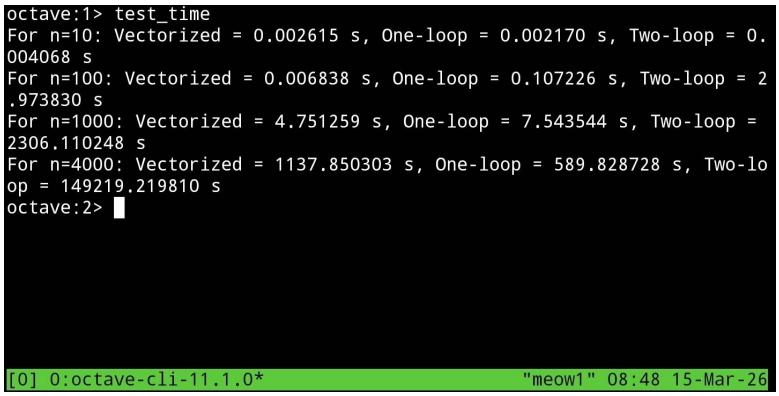

Figure 5: The results

(iv)

I found that the vectorized implementation significantly outperformed the looped versions because it avoids the interpreter overhead associated with element-by-element processing.

According to the Octave manual, vectorization allows the program to bypass high-level interpretation and utilize optimized internal C++, C, or Fortran libraries that leverage hardware vector instructions. The goal of vectorization is to write code that is making loops less (like for and while), instead, use whole array operations which is more efficient.

This approach achieved the 10X–100X speedup typical of Octave's matrix-oriented design. In contrast, my two-loop and one-loop versions were slower because Octave had to re-evaluate the code and check data types for every individual iteration. (one loop is faster than two-loop because the column addition also utilize the technique of vectorization)